

SME Financial Intelligence System

MVP v2 — Two-Sequence Workflow Report

Author: Tin Krupatiwittayakul | Date: 13 May 2026

This report documents the MVP refactor that aligns the system with the two-sequence design philosophy: **(1)** an external LLM designs entities from the raw uploaded data, capturing each entity's business *objective*, the department that owns it, and the data constraints; **(2)** a graph assembler turns those LLM blueprints into the interactive ontology graph the user can edit, while a memory layer keeps the design step accurate and cheap across re-runs.

1. Why the original workflow was wrong

In the previous MVP, ontology generation came *after* rule-based extraction. Entities were emitted by a hard-coded resolver (`entity_resolver.py`) whose only knowledge of the data was a fixed list of file-types and column aliases.

- **No business semantics.** Entities had a name but no objective, no department owner, no data constraints — so the graph could not be trusted as a description of *why* the data exists.
- **No room for the LLM to design.** The LLM was only called as a fallback for low-confidence column matches, never as the architect of the schema.
- **No user interaction.** The graph was display-only — no way to correct it or to draw a missing link between two entities.

2. The new two-sequence workflow

The corrected pipeline reverses the order of operations: the LLM designs first, then the rest of the pipeline consumes its output.

#	Stage	Owner	What happens
1	Upload	User	User drops Excel files into the dashboard.
2	LLM Entity Design	Claude API	For every file, the headers + 5 sample rows are sent to Claude. Claude returns a list of entities with objective, department, constraints, attributes, and likely relations.
3	Memory lookup	Backend	Before each LLM call the Design Memory layer checks a fingerprint of (file_type + sorted columns). On hit, the cached design is reused — no API call, no drift.
4	Graph Assembly	Backend	Designs are persisted as EntityDesign + EntityRelationDesign rows and rendered as a graph the user can edit.
5	User correction	User	User edits any node's objective / department / constraints; can draw a new relationship between two nodes; can re-run the LLM.
6	Approve & remember	User + Backend	On confirm, the design is written back to the memory layer so the next upload with the same fingerprint reuses it.
7	Record extraction	Backend	Standard column mapping, validation, scoring and journal posting run on the approved design.
8	Statements	Backend	Trial balance + income statement generated from the journal entries.

3. Components delivered in this release

3.1 Data model — new tables

Three new SQLAlchemy tables were added to backend/database.py:

Table	Purpose
EntityDesign	Schema-level entity blueprints produced by the LLM. Carries <i>objective</i> , <i>department</i> , <i>constraints</i> (JSON), <i>attributes</i> , <i>confidence</i> , <i>status</i> (proposed/approved/edited/rejected), and a node position.
EntityRelationDesign	Directed edges between EntityDesigns with label, cardinality, business objective and a status flag (proposed / user_added).
DesignMemory	Caches approved per-file designs by SHA-1 fingerprint of (file_type + sorted columns). Records hit_count + last_used so the system gets faster and more stable over time.

3.2 Services

File	Responsibility
services/entity_designer.py	Sequence-1 engine. Calls Claude with a strict-JSON prompt asking for entities with objective + department + constraints; falls back to a deterministic per-file_type design.
services/design_memory.py	Backend system that keeps accuracy: fingerprint → cached design. Exposes lookup(), remember(), stats().
services/graph_assembler.py	Sequence-2 engine. Persists EntityDesign + EntityRelationDesign rows and assembles the {nodes, edges, summary} graph that the dashboard renders.

3.3 API

Method	Path	Purpose
POST	/api/design/{period_id}/run	Sequence 1 — run the LLM designer over every file in the period.
GET	/api/design/{period_id}	Sequence 2 — return assembled {nodes, edges, summary} + memory stats.
PATCH	/api/design/entity/{design_id}	User edits any field on an EntityDesign — label, department, objective, constraints (JSON), position, status.
POST	/api/design/{period_id}/relation	User-drawn relationship between two existing EntityDesigns.
DELETE	/api/design/relation/{rel_id}	Remove a relationship.
POST	/api/design/{period_id}/approve	Mark all designs approved and write them into the memory layer.

3.4 Pipeline change

backend/api/pipeline.py now starts with a new *design* stage **before** any record extraction. For every file in the period, the LLM is asked to design entities (with memory lookup first). The merged design is persisted via graph_assembler.persist_design(). Only after the user approves does the rest of the pipeline (validate, score, journal-post, generate statements) execute.

3.5 Frontend interactivity

backend/dashboard.html — the graph inspector panel exposes every property of the new schema:

- Entity label, department dropdown, free-text **Objective** field, and JSON **Constraints** editor — saved via PATCH.
- A source-badge on each node tells the user whether this node came from *llm*, the *memory* cache, a *fallback*, or *manual edit*.
- A 'Link to...' button puts the canvas in **link mode**; clicking any other node creates a new EntityRelationDesign with prompts for label, cardinality and business reason.
- A **Re-design** tool-chip re-runs the LLM designer on demand.

4. Memory layer — accuracy mechanics

The memory layer is what turns the LLM's design step from a one-off guess into a reproducible, organisation-level decision.

Fingerprint formula:

```
sha1( lower(file_type) + '|' + '|'.join(sorted(lower(columns))) )
```

On a fresh upload the system:

- 1 Computes the fingerprint.
- 2 Tries `DesignMemory.lookup(user_id)`; falls back to the global cache.
- 3 On hit → returns the cached payload marked `source='memory'` and increments `hit_count`.
- 4 On miss → calls Claude; if Claude succeeds, the payload is held until the user confirms. Once confirmed, `DesignMemory.remember()` persists it under the fingerprint.

Net effect: the second time a user uploads the same monthly sales file format, the design step is instantaneous and identical to what they approved last time.

5. Trust, audit and override surface

Each `EntityDesign` row carries a *status* column with four values:

Status	When	Visual
proposed	LLM/memory returned it; user hasn't touched it.	Blue badge
edited	User changed objective/dept/constraints.	Purple badge
approved	User explicitly confirmed via <code>/approve</code> .	Green check
rejected	User dropped this entity from the design.	Red, hidden

Each `EntityRelationDesign` carries the same status plus a *source* column (llm | memory | user | fallback) so audit reports can answer the question *who decided this link existed?*

6. Files added / modified

File	Status	Lines
backend/database.py	modified	+72
backend/services/entity_designer.py	new	417
backend/services/design_memory.py	new	111
backend/services/graph_assembler.py	new	226
backend/api/entity_design.py	new	321
backend/api/pipeline.py	modified	+52
backend/api/ontology.py	modified	+24
backend/main.py	modified	+2
backend/dashboard.html	modified	+140

7. How to run the MVP locally

```
$ pip install -r requirements.txt $ export ANTHROPIC_API_KEY=sk-ant-... $ export DATABASE_URL=sqlite:///tmp/sme.db $ uvicorn backend.main:app --reload # Open http://localhost:8000
```

8. Try the new flow

- 1 Sign in, create a company, drop the sample Excel files from sample_data/ on the dashboard.
- 2 Watch the pipeline progress bar — a new **Sequence 1 — designing entities with LLM** step appears before extraction.
- 3 Click **Review ontology graph**. Each node now shows a department badge, an objective field, and a constraints JSON editor.
- 4 Click **Link to...** on a node, then click another node, and supply a relationship label — the new edge appears.
- 5 Press **■ Re-design** in the toolbar to re-run the LLM.
- 6 Press **Confirm & post journals** — the design is saved into the memory layer so next month's upload hits cache.

9. Known limitations / next steps

- The LLM fallback is deterministic but conservative; rare file shapes still benefit from a real API key.
- Drag-to-move nodes is wired through position_x/position_y but the dashboard SVG currently lays out via a fixed grid — full drag UX is the next frontend ticket.
- Memory is keyed on (file_type + sorted columns); future work: cluster on column *semantics* not just names so multilingual headers share a cache key.
- Editing the per-attribute list inside a node is read-only in this release — the JSON editor covers it via the constraints field.

End of report — SME Financial Intelligence System, MVP v2.